15-using-test-doubles-effectively/18/twitter_user_formatter/lib/twitter_user_formatter.rb

```
class TwitterUserFormatter
  def initialize(user)
    @user = user
  end

  def format
    @user.name + "'s website is " + @user.url
  end
end
```

The name and url methods come from an instance of the Twitter::User class. Constructing a real User requires multiple steps and different collaborators. It may be tempting to provide a fake implementation instead, like so:

15-using-test-doubles-effectively/18/twitter_user_formatter/spec/twitter_user_formatter_spec.rb

```
RSpec.describe TwitterUserFormatter do
  it 'describes their homepage' do
    user = instance_double(Twitter::User,
                           name: 'RSpec',
                           url: 'http://rspec.info')

    formatter = TwitterUserFormatter.new(user)

    expect(formatter.format).to eq("RSpec's website is http://rspec.info")
  end
end
```

This code would work well on earlier versions of the Twitter gem. Starting with version 5.0, though, the Twitter::User#url method returns a URI object instead of a plain string.

If we were to upgrade to the latest gem version and run this spec, it would still pass. Our code expects a string, and that's what our fake url method gives it.

Once we tried to use our TwitterUserFormatter in production, though, we'd start seeing exceptions. Specifically, when we try to concatenate the user's URL (which is now a URI instance) onto the description string:

```
@user.name + "'s website is " + @user.url
```

…we'd get a TypeError with the message no implicit conversion of URI::HTTPS into String.

This is the nightmare scenario for testing, where specs can give false confidence. Our test doubles were designed to mimic an interface that's not under our control, and that interface moved out from underneath us. (The Twitter gem maintainers are in fact careful about deprecations, but as developers we don't always remember to read the release notes.)

One way to reduce this risk is to use verifying doubles, as we've done for this example with an instance_double. These will detect when a class or method gets